# Building an acoustic diagnostics tool to optimise sound pressure level in enclosed spaces

Benjamin J Smith

<22029843>

https://github.com/uwe-scct/22029843_Digital_Systems_Project.git

UXCFXK-30-3

Digital Systems Project

UWE Bristol | University of the West of England

# Abstract

This project presents the development of an acoustic diagnostic tool designed to optimise sound pressure level (SPL) distribution in enclosed spaces. The study addresses the inherent trade-off in acoustic modelling, between computational efficiency and physical accuracy. It explores ray-based, image source based and wave-based models discovering and evaluating their respective limitations. Ultimately, to overcome these constraints, a hybrid model is proposed. This integrates geometrical and wave-based techniques to achieve the sought after balance between accuracy and performance, resulting in a system that contributes as a flexible and accessible tool for acoustic analysis.

# Acknowledgements

***This work is dedicated to the spiral***

***Special thanks to***

*Pascale Florence*

*Henry Smith*

*Kirsty Louise Smith*

*George Jacob Smith*

*Benjamin Alexander Joseph Webber*

*Dylan James Swinburne*

# Table of Contents

Abstract ..... 1
Acknowledgements ..... 2
Table of Contents ..... 3
Introduction ..... 5
1.1 Aims: ..... 5
1.2 Objectives: ..... 5
1.3 Scope: ..... 5
Literature Review ..... 6
Methodology ..... 8
3.1 Methodology overview ..... 8
3.2 System architecture ..... 8
Core components ..... 8
Data flow and integration ..... 9
Early Implementation and Development ..... 10
4.1 Early ray tracing implementation ..... 10
Model implementation ..... 10
Limitations ..... 10
4.2 Image Source Method (ISM) Implementation ..... 11
Model implementation ..... 11
limitations ..... 11
..... 12
4.3 Introducing Hybrid Models ..... 12
Model implementation ..... 12
SPL computation and visualization ..... 13
Advantages and limitations ..... 13
Final Design ..... 14
5.1 Overall architecture ..... 14
Physics module ..... 14
Speaker module ..... 14
GUI module ..... 14
5.2 Key features ..... 15
5.3 Limitations and advantages ..... 15
5.4 implementation ..... 16
Project Evaluation ..... 17

6.1 Evaluation overview ..... 17
6.2 Evaluating the modelling approaches ..... 17
6.3 Validation against theory ..... 18
6.4 System performance ..... 18
6.5 Usability and functionality ..... 18
6.6 Limitations ..... 18
6.7 Overall evaluation ..... 19
Glossary ..... 19
References / Bibliography ..... 20
Appendix A: First Appendix ..... 21
..... 21
Appendix B: full and final program ..... 22

# Introduction

Acoustic modelling plays a crucial role in the analysis of any space where sound behaviour is important, including architectural acoustics, industrial noise control, bass reflex enclosures, virtual simulation systems and many more. Accurately predicting and/or calculating how sound propagates through and around mediums is a complex problem involving both geometric propagation and wave-based phenomena such as diffraction or interference. Traditionally there are three main approaches to these problems: ray-based engines which offer computational efficiency but lack physical accuracy, Image Source methods which are more realistically accurate but assume unrealistic conditions and wave-based methods which provide highly accurate representations but are very computationally expensive. This trichotomy creates a practical requirement for acoustic modelling tools that balance accuracy and performance.

This project presents the development of an acoustic diagnostics tool that incrementally improves modelling accuracy through multiple stages. It begins with simple ray-tracing implementation to simulate basic sound propagation in complex or large enclosures, before then advancing into an Image Source Method (ISM) to cut on computation whilst increasing accuracy for specular and higher order reflection in 3D environments. Ultimately the final stage will explore a transition into a more complete 3D acoustic model, considering a plethora of wave phenomena whilst maintaining as much computational simplicity as possible. This staged approach will not only allow for comparative analysis between techniques but also simplify the implementation process, leading to a more accurate model.

## 1.1 Aims:

The project aims are to design and develop a personal acoustic diagnostic tool capable of simulating sound propagation in event spaces, using progressively more accurate modelling techniques. And to evaluate the effectiveness and limitations of ray-based, ISM, wave and hybrid approaches in acoustic simulation.

### 1.2 Objectives:

1. To implement a ray-tracing model for simulating sound propagation in an enclosed space.
2. To develop an image source method (ISM) for improved simulation of specular reflections.
3. To investigate the limitations of geometrical acoustic methods in representing real world sound behaviour.
4. To explore an extension toward a hybrid approach 3D acoustic modelling framework.
5. To visualise and output acoustic response based on different models.
6. And ultimately to produce a functional product.

## 1.3 Scope:

- The project focuses on geometrical acoustics methods.
- Wave based methods will be discussed, and implemented into geometrical model where applicable but not fully due to computational constraints.
- The system is designed purely for closed environments rather than outdoor acoustics.
- Diffraction and interference will be modelled approximately rather than fully.

The following chapter reviews existing acoustic modelling techniques and their literary history, providing a theoretical foundation for the development of this project.

# Literature Review

This project is an exploration into ray-based acoustic simulations in enclosed spaces; it hopes to give a specialised personal acoustics diagnostic tool that is optimised for event management. To do this I will explore fully ray-based, wave-based and hybrid approaches, in order to differentiate which practice best suits this task.

First, we will look at the early implementation of ray tracing in acoustics, notably Kokstad A. [1968] published his seminal paper "(c)alculating the acoustical room response by the use of a ray tracing technique" in the *Journal of sound and vibration* in 1968. They introduced a method for "simulating sound propagation by tracing a large number of discrete rays from a source to determine the impulse response and energy distribution…". This paper is regarded as foundational in computer aided acoustics, marking a shift toward new purely statistical methods like *Sabine's formula* (W. B. Joyce. [1974]) to simulation-based models. This early calculation of impulse response is based on ray generation, these rays are cast in all directions inside parameters and continue until the first point of collision with an interior surface, a secondary ray is then emitted from that point (Bojičić S., Pavlović D.S. and Mijić M. [2018]). The total "reflected" sound energy of the second ray from an interior surface is the sum of both specular and diffuse vectors from reflected sound energy. This can be described by Rindel's introduction of "vector modelling" which can be found in my appendix (Rindel J. H. [1995]). These two reflection vectors, both diffuse and specular, also define the direction of a reflected ray. The first is determined by Snell's law, while the latter is calculated by a random number generator using a probability function according to Lambert's law or sin(20) (Rindel J. H. [2004]) - diagrams from this extract can be found in the appendix. These early ray models allowed for efficient approximation of sound propagation in very complex spaces with minimal computational power.

The next noteworthy advancements in computer-based acoustic simulation models was the introduction of the Image Source Method or ISM. Kirszentien pioneered this practice with his journal "An Image Source Computer Model for Room Acoustics Analysis and Electroacoustic Simulation" (Kirszenstein, J. [1984]). ISM is defined as a foundational geometrical acoustic technique simulating specular reflections, primarily by modelling the early Room impulse Response (RIR). It operates on that "(w)hen a sound field emitted by a source is reflected at an acoustically-hard infinite flat surface, the reflected field is the same as the field produced by a source that is mirrored at that plane." (Siltanen, S., Lokki, T. and Savioja, L. [2010]). An ISM model assumes specular reflection and fails to consider diffuse reflection. However, because of this it allows for higher order reflections by creating image sources for existing image sources meaning many reflections with limited computation speed. Unfortunately, the number of image sources quickly explodes with exponentially increasing reflection order in unorthodox room shapes. However, in our use case, almost all event spaces are rectangular. This means an exact solution can be found, thus

allowing for this computational model to still be applicable.

On the other hand, in a true wave-based simulation, "the governing equations of wave motion in an enclosure are solved directly… yielding highly accurate schemes, but hampered by excessive computation times" (Jörgensson, P. and Kári, F. [2020]). This method has the highly attractive property of high-order accuracies, geometric flexibility and the ability to implement phase, constructive interference, destructive interference and air absorption (refraction), but in doing so becomes more resource intensive and therefore would be impossible to implement for a large or complex space.

In short, no one model is sufficient, presently the hybrid approach is widely used. By combining the ray tracing, image source methods (ISM) and wave-based methods (Vorländer M. [2013]). This defines surfaces in a described room as two numerical parameters: the absorption coefficient which quantifies the loss of energy after reflection, and the scattering coefficient which quantifies the diffusion of reflected sound energy (Kuttruff H. [2014]). Simply put, one would build a ray based engine and assign values to each ray based on which point of their wavelength cycle they are currently in, allowing for wave-based calculation without the computational requirements for simulating actual wave reflections, simultaneously the ISM model will handle reflection whilst holding accumulative attenuation and assigning a reflection coefficient to each surface. Kuttruff states that "(t)his method has the advantage that, once the model is at hand, many variations can be tried out with little expenditure: from the choice of various wall materials to changes in the shape of the room." He then emphasises the use of this model by stating that "(s)ince several properties of propagation are common to all sorts of waves, it is not absolutely necessary to use sound waves for the model tests" however he does go on to recognise that "techniques have reached a sufficiently high state… to generate and receive sound waves under the condition of propagation". Thus, a hybrid model paints a more accurate picture within our scope. Upon further research into this hybrid approach the literature led to the pioneer of said technique, Jens Holger Rindel. He stated in 1995 that "…room acoustics models have matured. Hybrid methods combine the best features from ISM and ray tracing methods and have led to significantly reduced calculation times" (Rindel J. H. [1995]).

In conclusion, this project should start with ray-based modelling for computational simplicity allowing for large or complex spaces, progress into ISM for deterministic reflections and higher accuracy and ultimately aim toward a full 3D acoustic model to capture the full spectrum of wave phenomena such as phase, interference and air absorption.

# Methodology

## 3.1 Methodology overview

The methodology adapted in this project follows a staged approach in implementing the acoustics diagnostics tool, with the objective of balancing efficiency and modelling accuracy. The system is designed to simulate sound propagation and then produce spatial representations of SPL. To achieve this multiple modelling techniques will be explored sequentially. First of a ray tracing model will be implemented, progressing toward the ISM method and culminating into a hybrid approach that integrates the strength of both geometrical and wave-based methods. This iterative strategy enables both comparative evaluation and continual improvement of accuracy.

This methodology is divided into three main components. First, the introduction of the ray tracing model for initial approximation of sound propagation, offering high computational efficiency but fairly low accuracy. Secondly, the ISM extends the modelling capabilities to accurately emulate specular reflection, enhancing the simulated impulse response. Finally, the hybrid approach addresses the inherit limitations of purely geometric methods, incorporating wave-based considerations to improve SPL prediction and spatial resolution. Once the model is at a sufficient level, user-defined parameters will be introduced providing scenario specific analysis.

Evaluation of the methodology will be performed through comparative analysis of each model's outputs, including heat map distributions and qualitative assessment of reflection behaviour. This approach assures that methodological decisions are justified both theoretically and practically, providing a complete and robust framework for the development and validation of the acoustic diagnostic tool.

## 3.2 System architecture

The tool is designed as a modular system, with distinct components corresponding to input handling, acoustic modelling, SPL calculation and output visualisation. This architecture supports the staged development approach, allowing for each technique to be implemented and evaluated independently before integration. The modular design, in separation of the physics and GUI engines, facilitates user interaction enabling parameter adjustments without the modification of underlying computational engine.

### Core components

*Input module:*

Once the simulation is at a sufficient level the system will accept user-defined parameters including frequency, direction angle, reflection order, listener height, room dimensions and wall materials

*Acoustic modelling engine*

This is the central module that performs the core simulation tests and contains three main submodules:

- **Ray-tracing:** simulates basic sound propagation and initial SPL approximation.
- **Image source Method:** calculates the specular reflections using recursively generated image sources.
- **Wave-based additions:** adds approximations for wave-based behaviour.

*SPL calculation module*

This component compiles contributions from direct and reflected paths to iteratively compute the total SPL at each point on the spatial grid. Air absorption, distance decay, accumulative attenuation and reflection coefficients are all incorporated into the calculations.

*Visualisation module*

Simulation results compiled by the SPL module are rendered as heatmaps, representing the SPL distributions across the enclosure. This visual output provides the spatial insight for comparative analysis and therefore will be present in all iterations of the model.

### Data flow and integration

Data flows sequentially from user input through acoustic modelling to SPL calculation and finally to visualisation. Each module operates independently but will communicate via well-defined data structures, ensuring reproducibility and flexibility for future enhancements. This framework also allows for each modelling technique to be benchmarked under identical conditions.

# Early Implementation and Development

## 4.1 Early ray tracing implementation

The ray tracing model represents the initial stage of the project, providing a computationally efficient approximation of sound propagation within an enclosed space. Rays are emitted from a defined source, representing the speaker, and propagate in linear paths across a 2D plane. Each ray has parameters for energy which is attenuated according to a vector of distance and direction.

### Model implementation

The enclosure or room is defined as a uniform grid; this grid has a resolution parameter to balance computational cost with spatial accuracy. The speaker is defined at a user-defined location and height, with a directional angle relative to the room's axes. Directional attenuation is applied using a cosine-based directivity model, simulating the decreased sound intensity behind the speaker, which can be observed in the output. To avoid singularities at the source location a small offset is included in the distance calculation.

*Figure 1: early implementation of ray-tracing model*

```python
import numpy as np
import matplotlib.pyplot as plt

# --- Room setup ---
room_length = 7.0   # meters (X)
room_width = 5.0    # meters (Y)
speaker_height = 1.2  # meters
grid_res = 0.05  # grid resolution in meters

x = np.arange(0, room_length, grid_res)
y = np.arange(0, room_width, grid_res)
X, Y = np.meshgrid(x, y)

# --- Speaker setup ---
speaker_pos = np.array([2.5, 2.0, speaker_height])  # (x, y, z)
direction_angle = np.deg2rad(45)  # degrees -> radians, 0° faces +x

# --- Compute directionality ---
dx = X - speaker_pos[0]
dy = Y - speaker_pos[1]
dist = np.sqrt(dx**2 + dy**2) + 1e-2 # avoid division by zero

# Angle between speaker direction and point
theta = np.arctan2(dy, dx) - direction_angle
directivity = np.cos(theta)
directivity[directivity < 0] = 0.05  # back of speaker = no sound

# --- Simple sound pressure model ---
p = directivity / dist  # 1/r decay
p /= np.max(p)  # normalize to max = 1

# Convert to SPL (0 dB = max pressure)
spl = 20 * np.log10(p + 1e-6)

# --- Plot heat map ---
plt.figure(figsize=(8, 5))
plt.imshow(spl, extent=(0, room_length, 0, room_width), origin='lower', cmap='inferno')
plt.colorbar(label='Relative SPL (dB)')
plt.scatter(speaker_pos[0], speaker_pos[1], c='cyan', s=60, label='Speaker')
plt.title('Simulated Sound Pressure Heat Map (Simplified)')
plt.xlabel('Room length (m)')
plt.ylabel('Room width (m)')
plt.legend()
plt.show()
```

Sound pressure levels are approximated using an inverse distance law, with contributions normalised to the maximum value for clean visualisation. The resulting data from this is then rendered as a two-dimensional heat map, providing visual representation of SPL distribution across the room. This approach allows rapid assessment of source placement and directional effects and serves as a baseline for more advanced modelling techniques.

### Limitations

Whilst being very efficient, the ray tracing model assumes free-field environments with line-of-sight propagation and fails to account for reflection, diffraction or interference effects (as seen in *figure 2*). Furthermore, the directional attenuation is simplified, and energy decay is modelled purely geometrically, limiting the physical accuracy at low frequencies.

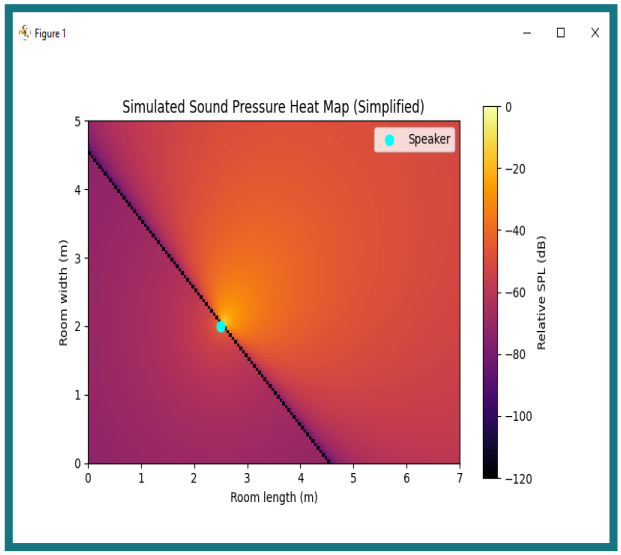



Figure 2: ray-tracing model heatmap output

## 4.2 Image Source Method (ISM) Implementation

The image Source Method or ISM represents the completion of the second stage of acoustic modelling in this project, extending the ray-based approach into 3D specular reflections. Unlike the initial ray-based model, ISM generates a set of virtual sources corresponding to image reflections from the room boundaries, this allows the model to capture early reflections and improve SPL prediction accuracy.

### Model implementation

The enclosure is now defined as a 3D grid, with each coordinate representing a point at which SPL is calculated. The speaker is positioned at a user defined location with direction relative to room axes. The function generate_image_sources_3D recursively generates image sources for every surface (wall, floor and ceiling), up to a maximum reflection order defined by the user to help stop recursion issues. Each image source inherits an attenuation factor corresponding to the reflection coefficient of the boundary, thereby simulating energy loss during reflection.

At each grid point, both the direct and reflected sources are computed by use of an inverse-distance law combined with a cosine-based directional attenuation. All contributions from all sources are summed and normalised to produce relative SPL values, which are then converted to decibel scale for visualisation.

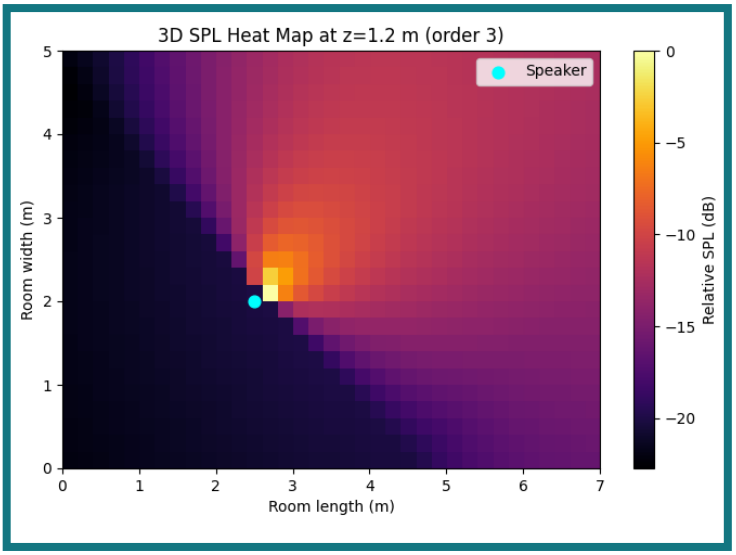
*Figure 3: ISM model implementation*

```python
import numpy as np
import matplotlib.pyplot as plt

# Room setup
room_length = 7.0   # meters X
room_width = 5.0    # meters Y
room_height = 3.0   # meters Z
speaker_height = 1.2
grid_res = 0.2      # coarser resolution for 3D in meters

# Reflection parameters
reflection_coeff = 0.24 # 0 = totaly absorbant, 1 = totaly reflective, brick wall is 0.18 - 0.36
max_order = 3 # number of reflections (3 works best)

# --- 3D Grid ---
x = np.arange(0, room_length, grid_res)
y = np.arange(0, room_width, grid_res)
z = np.arange(0, room_height, grid_res)
X, Y, Z = np.meshgrid(x, y, z, indexing='ij')  # shape (Nx, Ny, Nz)

# --- Speaker ---
speaker_pos = np.array([2.5, 2.0, speaker_height])
direction_angle = np.deg2rad(45)  # 0° faces +x

# --- Pressure computation ---
def compute_pressure_3d(source_pos, source_angle, attenuation=1.0):
    dx = X - source_pos[0]
    dy = Y - source_pos[1]
    dz = Z - source_pos[2]
    dist = np.sqrt(dx**2 + dy**2 + dz**2) + 1e-2  # full 3D distance

    # simplified planar directivity
    theta = np.arctan2(dy, dx) - source_angle
    directivity = np.cos(theta)
    directivity[directivity < 0] = 0.0

    return attenuation * directivity / dist

# Function that generates image sources (pixels) recursively
def generate_image_sources_3d(pos, angle, order):
    if order == 0:
        return [(pos, angle, 1.0)]

    images = []
    # Reflect walls x
    images.append((np.array([-pos[0], pos[1], pos[2]]), np.pi - angle, reflection_coeff))
    images.append((np.array([2*room_length - pos[0], pos[1], pos[2]]), np.pi - angle, reflection_coeff))
    # Reflect walls y
    images.append((np.array([pos[0], -pos[1], pos[2]]), -angle, reflection_coeff))
    images.append((np.array([pos[0], 2*room_width - pos[1], pos[2]]), -angle, reflection_coeff))
    # Reflect floor/ceiling z
    images.append((np.array([pos[0], pos[1], -pos[2]]), angle, reflection_coeff))
    images.append((np.array([pos[0], pos[1], 2*room_height - pos[2]]), angle, reflection_coeff))

    if order > 1:
        higher_order_images = []
        for img_pos, img_angle, atten in images:
            higher_order_images += generate_image_sources_3d(img_pos, img_angle, order-1)
        images += higher_order_images

    return images

# --- Compute SPL ---
p_total = compute_pressure_3d(speaker_pos, direction_angle)  # direct sound
all_images = generate_image_sources_3d(speaker_pos, direction_angle, max_order)

for img_pos, img_angle, atten in all_images[1:]:  # skip direct
    p_total += compute_pressure_3d(img_pos, img_angle, attenuation=atten)

# Normalize and convert to SPL
p_total /= np.max(p_total)
spl = 20 * np.log10(p_total + 1e-6)

# --- Visualization: horizontal slice at listener height ---
listener_height = 1.2
z_index = np.argmin(np.abs(z - listener_height))

plt.figure(figsize=(8, 5))
plt.imshow(spl[:, :, z_index].T, extent=(0, room_length, 0, room_width),
           origin='lower', cmap='inferno', aspect='auto')
plt.colorbar(label='Relative SPL (dB)')
plt.scatter(speaker_pos[0], speaker_pos[1], c='cyan', s=60, label='Speaker')
plt.title(f'3D SPL Heat Map at z={listener_height} m (order {max_order})')
plt.xlabel('Room length (m)')
plt.ylabel('Room width (m)')
plt.legend()
plt.show()
```

### limitations

the ISM provides a significant improvement over the simplistic ray tracing model by accurately modelling specular reflections and enabling 3D analysis of an acoustic field (as seen in *figure 3*). This approach allows for analysis of source placement, reflection order and material properties' (through reflection coefficient) effects on SPL distribution. However, ISM does still assume Idealised reflection and cannot account for complex interference effects.

*Figure 3: Heatmap of SPL distribution output from ISM model (mark_4.py)*




## 4.3 Introducing Hybrid Models

The hybrid modelling approach represents the start of the final stages of this acoustic simulation project, combining the strengths of geometrical techniques like ray-casting and ISM with wave-based considerations to more accurately predict sound propagation within enclosed spaces. Building on the previous models the hybrid model incorporates complex pressure calculations, including phase, constructive interference, destructive interference and air absorption effects to capture both direct and reflected sound fields with higher fidelity.

### Model implementation

The hybrid model extends the ISM framework by representing each source, including the ones recursively generated as a complex pressure wave. The amplitude of each contribution is computed using the same inverse-distance law but also now considers directivity and a phase term proportional to the propagation distance. Furthermore, air absorption is incorporated through the use of an exponential attenuation factor, reflecting the frequency dependent level of dampening as sound travels through air. This implementation can be found in the compute_pressure_3D_complex function, where the pressure contribution of each source is calculated at all grid points.

*Figure 4: first hybrid model implantation*

```
import numpy as np
import matplotlib.pyplot as plt

# wavelength constants
c = 343.0 # speed of sound in m/s
frequency = 200.0 # Hz
wavelength = c / frequency
k = 2 * np.pi / wavelength # wave number

# Room setup
room_length = 10.0   # meters X
room_width = 10.0    # meters Y
room_height = 3.0   # meters Z
speaker_height = 1.2
grid_res = 0.2     # coarser resolution for 3D in meters

# Reflection parameters
reflection_coeff = 0.7 # 0 = totaly absorbant, 1 = totaly reflective
max_order = 2 # number of reflections (3 works best)

# --- 3D Grid ---
x = np.arange(0, room_length, grid_res)
y = np.arange(0, room_width, grid_res)
z = np.arange(0, room_height, grid_res)
X, Y, Z = np.meshgrid(x, y, z, indexing='ij')  # shape (Nx, Ny, Nz)

# --- Speaker ---
speaker_pos = np.array([2.0, 2.0, speaker_height])
direction_angle = np.deg2rad(45)  # 0° faces +x
```

Recursive image source generation is maintained from the older ISM, allowing multiple reflections up to the previously mentioned user-defined reflection order. However, now each reflected source is assigned an attenuation factor based on the boundaries' reflection coefficient, and phase inversion is applied where

applicable. This allows for different materials to be emulated and therefore analysed. The ISM function now implements this new recursion, ensuring that higher order reflections are systematically included in field calculations.

## SPL computation and visualization

The magnitude of the complex pressure field is extracted to produce SPL representation, converting physical pressure values to a logarithmic decibel scale. A horizontal slice at listener height is rendered as a heat map similarly to the previous model but now interference patterns resulting from the superposition of the direct and reflected waves are also accounted for. This approach allows for analysis of constructive and destructive interference providing a more realistic output (as seen in *figure 5*).

## Advantages and limitations

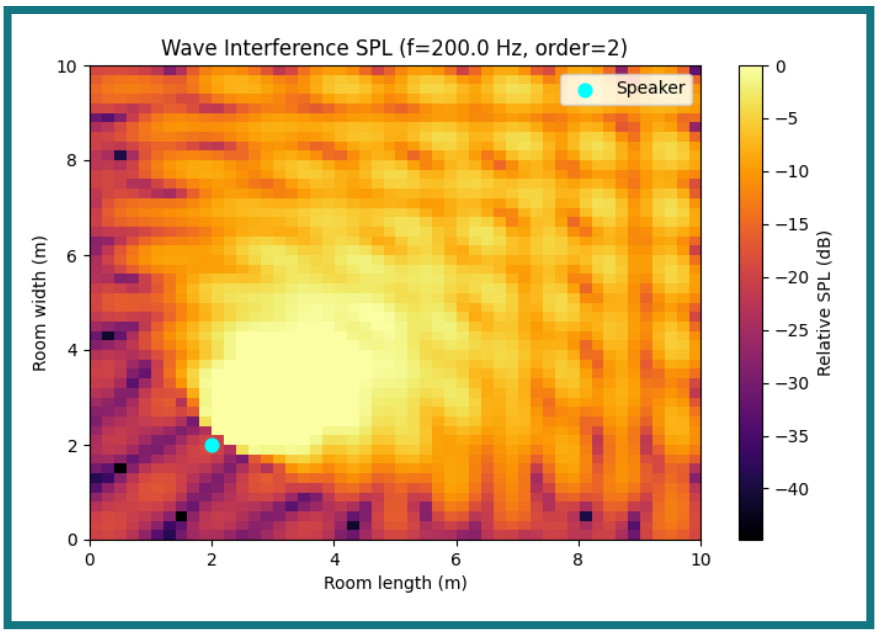
By integrating wave-based effects into the geometrical reflection model, the hybrid approach overcomes many of the inherent accuracy limitations that ray tracing and ISM models present. It can capture early reflection, wave interference and phase-dependant effects - this addresses the low frequency failures of the previous models. However, computational complexity remains far higher than purely geometrical models, and the hybrid model still relies on simplified assumptions regarding reflection coefficients and uniform air absorption. Despite these limitations, the model constitutes a practical and flexible tool for analysing SPL distribution and optimising acoustic performance in enclosed spaces, therefore satisfying most of the project's requirements so far.

```python
# Function that generates image sources (pixels) recursively with keeping attenuation
def generate_image_sources_3d(pos, angle, order, current_attentuation=1.0):
    if order == 0:
        return [(pos, angle, current_attentuation)]

    images = []

    def reflect(new_pos, new_angle):
        phase_flip = -1
        return (new_pos, new_angle, current_attentuation * reflection_coeff * phase_flip)

    images.extend([
        reflect(np.array([-pos[0], pos[1], pos[2]]), np.pi - angle),
        reflect(np.array([2*room_length - pos[0], pos[1], pos[2]]), np.pi - angle),
        reflect(np.array([pos[0], -pos[1], pos[2]]), -angle),
        reflect(np.array([pos[0], 2*room_width - pos[1], pos[2]]), -angle),
        reflect(np.array([pos[0], pos[1], -pos[2]]), angle),
        reflect(np.array([pos[0], pos[1], 2*room_height - pos[2]]), angle)
    ])

    if order > 1:
        higher = []
        for p, a, att in images:
            higher += generate_image_sources_3d(p, a, order-1, att)
        images += higher

    return images
# --- Pressure computation --- NEW!!!
def compute_pressure_3d_complex(source_pos, source_angle, attenuation=1.0):
    dx = X - source_pos[0]
    dy = Y - source_pos[1]
    dz = Z - source_pos[2]

    distance = np.sqrt(dx**2 + dy**2 + dz**2) + 1e-6  # full 3D distance

    dir_vector = np.stack((dx, dy, dz), axis=-1)
    dir_unit = dir_vector / distance[..., np.newaxis]

    forward = np.array([np.cos(source_angle), # x
                        np.sin(source_angle), # y
                        0.0]) # z - for tilt implementation later

    cos_angle = (dir_unit[..., 0] * forward[0] +
                 dir_unit[..., 1] * forward[1] +
                 dir_unit[..., 2] * forward[2] )

    directivity = np.clip(cos_angle, 0.0, None) **2

    # Phase term
    phase = np.exp(1j * k * distance)

    # air absorption
    air_absorption = np.exp(-0.01 * distance)

    # Complex pressure field
    return attenuation * directivity * phase *air_absorption / distance
# --- Compute total complex pressure ---
p_total = np.zeros_like(X, dtype=complex)

all_sources = generate_image_sources_3d(speaker_pos, direction_angle, max_order)

for pos, angle, atten in all_sources:
    p_total += compute_pressure_3d_complex(pos, angle, atten)

# --- Convert physical pressure ---
p_mag = np.abs(p_total)

# Convert to SPL-like dB
spl = 20 * np.log10(p_mag + 1e-12)
spl = np.clip(spl, -60, 0)

# --- Plot horizontal slice ---
listener_height = 1.2
z_index = np.argmin(np.abs(z - listener_height))

plt.figure(figsize=(8, 5))
plt.imshow(spl[:, :, z_index].T,
           extent=(0, room_length, 0, room_width),
           origin='lower',
           cmap='inferno',
           aspect='auto')

plt.colorbar(label='Relative SPL (dB)')
plt.scatter(speaker_pos[0], speaker_pos[1], c='cyan', s=60, label='Speaker')

plt.title(f'Wave Interference SPL (f={frequency} Hz, order={max_order})')
plt.xlabel('Room length (m)')
plt.ylabel('Room width (m)')
plt.legend()
plt.show()
```

*Figure 4: first hybrid model implantation*

*Figure 5*

*Outputted heatmap of SPL level at 200hz and 2 orders of reflection from hybrid model (Mark_7.py)*




# Final Design

The completed acoustic diagnostic tool integrates the previously developed hybrid framework with an interactive graphical user interface (GUI), providing a flexible, user-centric system for analysing SPL distribution. The system combines accurate wave-based simulation with geometrical reflection modelling, allowing users to explore the impact of placement, frequency, material properties and room dimensions on acoustic performance in real time.

## 5.1 Overall architecture

This too adapts the modular architecture, separating the core physics from the GUI and interaction logic:

### Physics module

This module contains functions for

- generating a three-dimensional spatial grid ( generate_grid ).
- computing complex pressure fields ( compute _pressure_3D_complex ).
- generating recursive image sources ( generate_image_sources_3d ).
- and calculating total SPL ( compute field ).

these functions maintain compatibility with the multiple modelling approaches (ray tracing, ISM and wave-based).

### Speaker module

This module is encapsulated inside the speaker class which stores the position, orientation and graphical representation of the source/speaker.

### GUI module

This module is implemented via the AcousticGUI class, which handles visualisation, user input and real-time updating of SPL heatmaps. It incorporates sliders, text boxes and radio buttons to control

frequency, source orientation, reflection order, listener height, room dimensions and material properties. It also connects the speaker to mouse events for intuitive spatial adjustments.

The separation of these modules ensures maintainability, facilitates further extension and clearly distinguishes between computational and interactive components.

*Full structure can be found in appendix A (figure 12).*

## 5.2 Key features

The final iteration introduces several new features compared to previous prototypes:

1. **Interactive heat map visualization**:
   - Real-time plotting of SPL distribution using Matplotlib.
   - Heat map updates dynamically as users adjust parameters, providing immediate feedback for analysis.
2. **User-controlled parameters:**
   - Frequency slider: adjusts frequency output from source, affecting wavelength and interference patterns.
   - Angle slider: rotates the source orientation to better simulate directional speakers.
   - Reflection order slider: controls the number of recursive reflections included.
   - Listener height slider: selects horizontal slice for SPL visualization.
   - Room dimensions text boxes: dynamically updates room size, automatically recalculating the grid.
   - Material radio buttons: adjust wall reflection coefficients for different materials.
3. **Dynamic speaker manipulation:**
   - Drag-and-drop functionality allows the user to move the source interactively.
   - SPL heat map updates immediately, illustrating the impact of source location in real-time.
4. **Performance optimization:**
   - Coarse grid resolution and attenuation thresholds are used to reduce unnecessary computation for low-contributing image sources.
   - Updates are throttled during drag events ( update_interval ) to maintain GUI responsiveness.

## 5.3 Limitations and advantages

The final design of this acoustic diagnostic tool demonstrates a balance between modelling accuracy, computational efficiency and usability. One of the key advantages of the system that allows it to provide this balance is its use of the hybrid model. This model integrates geometrical reflections with wave-based effects, resulting in more accurate SPL distribution. It is further enhanced by the interactive GUI, allowing real time adjustments of a plethora of variables, thereby supporting the intuitive exploration of acoustic behaviour. Additionally, the modular architecture produced by the project's methodology improves maintainability and extensibility.

However, as is inherent when balancing computational simplicity, several limitations remain. The model still relies on simplified assumptions in some cases, including uniform reflection coefficients and directivity functions, this could reduce accuracy, especially in complex and or irregular

environments. Due to the new responsive heat mapping computational performance is constrained by grid resolution and reflection order, requiring trade offs between simulation detail and speed. Finally, although the model takes into account wave-based assumptions, it does not fully implement physical wave-based methods limiting its usage in low frequency events involving diffraction. But, despite these limitations, the system does provide a practical and computationally efficient tool for visualising acoustic behaviour in an enclosed space.

## 5.4 implementation

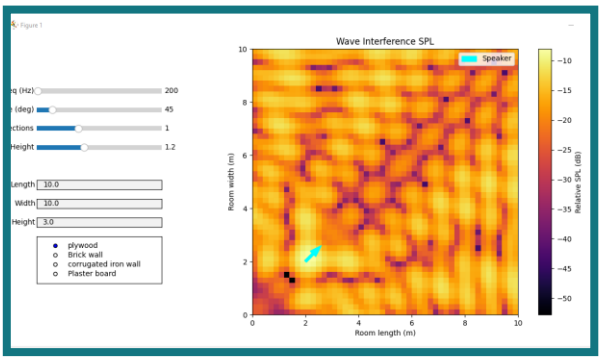



*Figure 6*

Mark_12.py output of SPL visualization

This shows the app upon launch with more accurate sound behavior depiction than previous models.

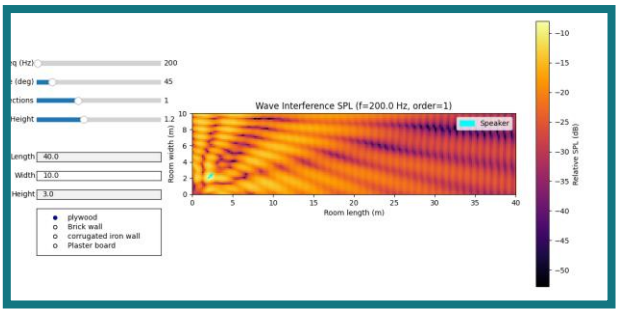



*Figure 7*

Mark_12.py output of SPL visualization with altered room length

By altering the text box entry not only does the simulation resize but the matplotlib graph and window dynamically resize as well.

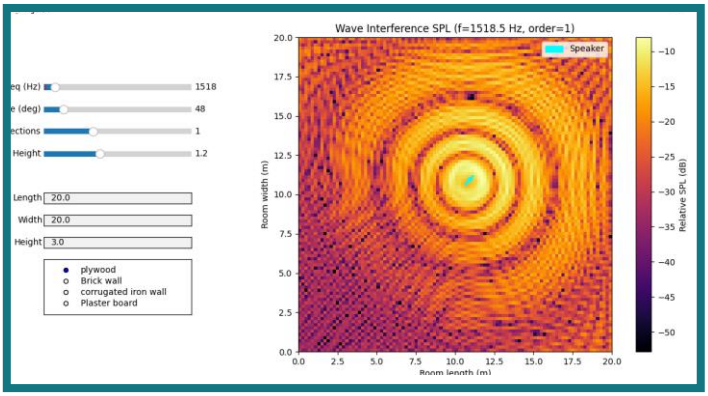



*Figure 8*

Mark_12.py output of SPL visualization with larger room size and altered frequency

This figure shows a 20x20m room at a higher frequency where sound propagation is more complex.

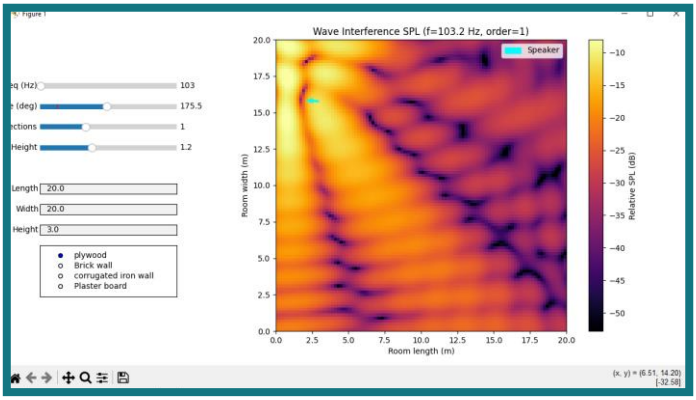



*Figure 9*

Mark_12.py output of SPL visualization with altered frequency and direction angle

Dark spots of SPL prove directivity.

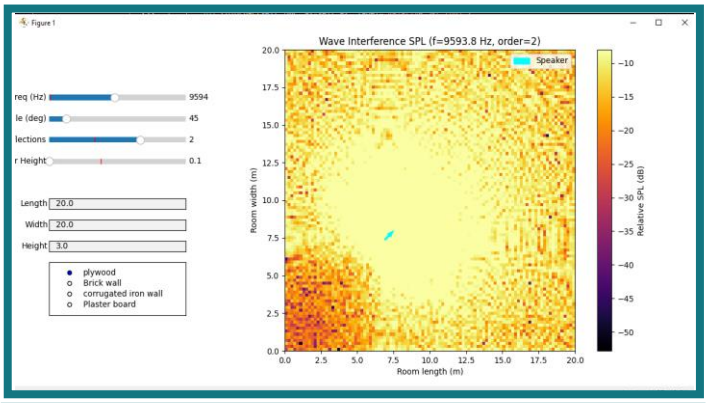



*Figure 10*

Mark_12.py output of SPL visualization at high frequency with high reflection order

As shown in *figure 7* although SPL at high frequency can seem chaotic it is indeed patterned and still displaying dead spots of SPL.

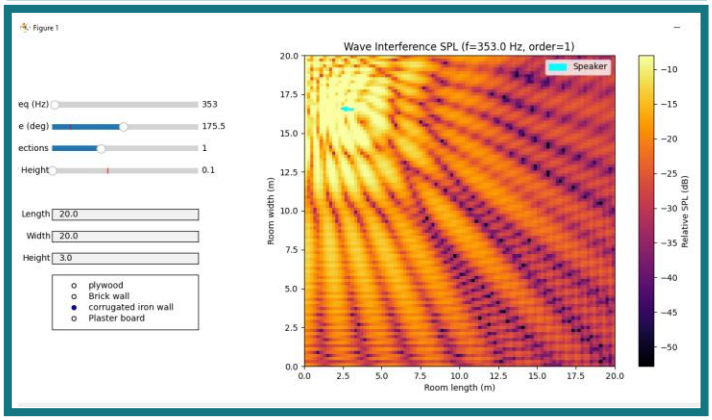



*Figure 11*

Mark_12.py output of SPL visualization with altered reflection coefficients

Figure 8 presents the use of the radio buttons changing the emulation of the walls' base material, increasing their reflectivity and therefore increasing the overall SPL.

***The Full program can be found in Appendix B or on the GitHub linked in both the appendix A and the front page.***

# Project Evaluation

## 6.1 Evaluation overview

This section evaluates the effectiveness of the developed acoustic diagnostic tool, in its aim of optimising SPL distribution in enclosed spaces. The evaluation will consider three main aspects: the accuracy and suitability of implemented modelling techniques, the computational performance of the system and the usability of the interactive tool. These three areas will be assessed in the context of both the literature review, methodology and introduction to this project.

## 6.2 Evaluating the modelling approaches

The initial ray-based model, as previously mentioned, provided a computationally efficient method for approximating sound propagation. This model did successfully demonstrate some fundamental behaviours such as distance attenuation and directional spreading. However, as defined in the literature, ray-based approaches inherently neglect many wave-based phenomena; resulting in unrealistic SPL distributions that lack interference patterns.

The next implementation introduced the Image Source Method, introducing specular reflection and therefore improving physical realism. This aligned well with the established research, which recognises ISM as an effective technique for early reflection modelling. The recursive image source generation allows for higher order reflections, producing more spatially accurate SPL distributions. Nevertheless, the method remains limited in its assumption of perfect

reflection and can massively increase computational cost with increase reflection order.

The introduction of wave-based modelling can be seen as of Mark_7 through complex pressure calculations, marking a substantial improvement in accuracy. By incorporating phase information, the model is able to simulate interference patterns. This is directly reflected in the literature, which emphasises the importance of wave-based methods in capturing low-frequency behaviour and modal effects. Even so the model represents an implementation of simplified assumptions rather than a full numerical wave solution and therefore does not fully capture diffraction or complex boundary interactions.

Finally, the hybrid approach successfully combines the strengths and weaknesses of both geometrical and wave-based methods. It achieves the sought after balance between computational efficiency and physical accuracy, further supporting the hypotheses found in the literature, that hybrid models offer a practical compromise for real-world applications.

## 6.3 Validation against theory

The behaviour of the final model is consistent with fundamental acoustic principles discovered in the review. The inverse-distance attenuation and logarithmic SPL scaling produce the anticipated decay patterns. The introduction of reflections also increased the SPL near boundaries. The further inclusion of phase leads to observable interference, particularly at low frequencies (as seen in *figure 5*). These observations are consistent with findings from the literature review.

## 6.4 System performance

The key consideration throughout the development of this project was the trade-off between accuracy and computational performance. The use of the 3D grid enables spatial analysis but introduces significant computational overhead, especially when utilising higher resolutions. Similarly increasing reflection order improves realism but results in exponential growth of the number of image sources and therefore computational requirements. To address this, several optimisation strategies were implemented, including reflection attenuation thresholds and update throttling during GUI updates. These measures successfully maintained real-time interactivity while attempting to preserve levels of accuracy. However, having the system constrained by these parameters will unfortunately hinder accuracy particularly for high frequency simulations (as seen in figure 10).

## 6.5 Usability and functionality

The interactive GUI demonstrates strong usability in the final product, features such as real-time parameter adjustments, drag and drop speaker positioning and dynamic heatmap visualisation significantly enhance user accessibility. The features enable flexible experimentation and analysis of acoustic patterns. The level of interactivity represents a key strength in this project, going beyond the objectives set out in the introduction. They translate the complex acoustic modelling into an intuitive and accessible format, supporting the tool's intended diagnostic purpose.

## 6.6 Limitations

Despite its strengths, the system does still have several limitations. For example, the use of uniform reflection coefficients simplifies material behaviour and does not account for frequency-dependent absorption, and the simplified directivity model does not represent real-world speaker characteristics in perfect detail. Furthermore, while wave-based effects are partially incorporated, the model does not implement a full wave-based solution (to save on computational efficiency). As a result of this, phenomena such as diffraction and scattering are only partially represented.

## 6.7 Overall evaluation

Overall, the project successfully achieves its aim of developing an acoustics diagnostic tool capable of modelling and visualising SPL distributions in enclosed spaces. Moreover, the progression from ray-based to ISM and ultimately a hybrid model, reflects that a clear and logical progression was achieved, supporting the literature review and methodology.

The final system demonstrates an effective balance between accuracy, performance and usability, making it a practical tool for exploratory acoustic analysis. And while some limitations remain the project stands as a strong base for further development and extension.

# Glossary

**Acoustic ray tracing -** is a geometrical modelling technique that simulates sound propagation by tracing paths of energy particles (rays) from a source to a receiver.

**impulse response -** a, typically digital, recording that captures how an acoustic environment react to a very short full frequency sound.

**Sabines' formula -** an empirical acoustic equation that calculates a rooms reverberation time (rT60).

**rT60 –** a calculated metric derived from IR data to quantify how quickly sound decays.

**Attenuation –** the total reduction in energy of an acoustic wave as it propagates through a medium.

**specular reflection -** Specular reflection is the mirror-like reflection of light (or other waves, in this case sound) from a smooth, polished surface, where rays reflect in a single direction.

**Acoustically hard –** a material that reflects the vast majority or all of the sound energy that strikes it.

**diffuse reflection -** the scattering of waves in many directions when striking a rough or uneven surface.

**Snells' law -** an equation that dictates how waves bend when crossing the boundary between media.

**Lamberts' law -** part of the beer lambert law is an equation to quantify the exponential loss of energy as a wave travels through a uniformly absorbing medium.

**Vector -** a mathematical or physical quantity that has both magnitude and direction.

**ISM -** The Image-Source Method is a geometric acoustic algorithm used to simulate sound propagation and reflections in an enclosed space by creating virtual "image sources" reflected across wall planes.

**RIR -** the same as Impulse Response, acts as a digital fingerprint.

**Reflection Order -** defines the number of surfaces a sound wave hits before it reaches a receiver.

**Scattering Coefficient -** quantifies the ratio of non-specularly reflected acoustic energy to the total reflected energy from a surface, allowing us to simulate surface roughness.

**Phase** – describes the position of a sound wave at a specific point in time within its cyclic journey, often and in this case measured in degrees 0-360.

**Destructive interference** – when two sound waves line up so that their peaks and troughs overlap, causing reduction or total cancellation of amplitude.

**Constructive interference -** when two sound waves perfectly align their peaks, resulting in a wave with a higher amplitude.

# References / Bibliography


Krokstad, A., Strom, S. and Sørsdal, S. (1968) Calculating the Acoustical Room Response by the Use of a Ray Tracing Technique. Journal of Sound and Vibration [online]. 8 (1), pp. 118-125. [Accessed 29 March 2026].

Joyce, W.B. (1974) The Journal of the Acoustic Society of America. Sabine's Formula [online]., p. 812. [Accessed 30 March 2026].

Vorländer, M. (2013) The Journal of the Acoustic Society of America. Computer Simulations in Room Acoustics: Concepts and Uncertainties [online]. 133 (3), pp. 1203-1213. [Accessed 30 March 2026].

Kuttruff, H. (2014) 5th Edition. Room Acoustics [online]. 5, pp. 316-331. [Accessed 31 March 2026].

Rindel J. H. (1995) Computer Simulation Techniques For Acoustical Design of Rooms. [online]. 23, pp. 81-86. [Accessed 31 March 2026].

Rindel, J.H. (2004) Odeon and Scattering Coefficient. [online]. 1, pp. 1-45. [Accessed 31 March 2026].

Bojičić S., Pavlović D.S. and Mijić M. (2018) Archives of Acoustics. Formation of Scattering Characteristics For Acoustical Ray Tracing Simulation [online]. 43, p. 4. [Accessed 31 March 2026].

Kirszenstein, J. (1984) Applied Acoustics. An Image Source Computer Model For Room Acoustics Analysis and Electroacoustic Simulation [online]. 17 (4), pp. 275-290. [Accessed 31 March 2026].

Siltanen, S., Lokki, T. and Savioja, L. (2010) Proceedings of the International Symposium on Room Acoustics. Rays Or Waves? Understanding the Strengths and Weaknesses of Computational Room Acoustics Modeling Techniques [online]., pp. 1-6. [Accessed 31 March 2026].

Jörgensson, P. and Kári, F. (2020) Wave-based Virtual Acoustics. [online]., pp. 8-76. [Accessed 01 April 2026]

SynAudCon(1996). Available from: https://www.prosoundtraining.com/2024/03/15/reverberation-time-rt/ [Accessed 01 April 2026]

OnlineMathLearning(2005). Available from: https://www.onlinemathlearning.com/specular-reflection.html [Accessed 01 April 2026]

Photokonnexion(2013). Available from: https://www.photokonnexion.com/definition-diffuse-reflection/ [Accessed 01 April 2026]

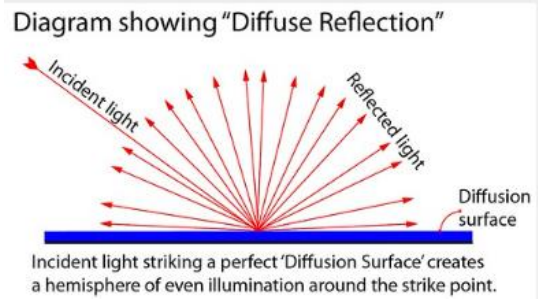
PersonalMath(N/A). Available from: https://personal.math.ubc.ca/~cass/courses/m309-01a/chu/Fundamentals/snell.htm [Accessed 01 April 2026]

# Appendix A: First Appendix

https://github.com/uwe-scct/22029843_Digital_Systems_Project.git

**Diffuse Reflection diagram:**

*(Photokonnexion[2013])*

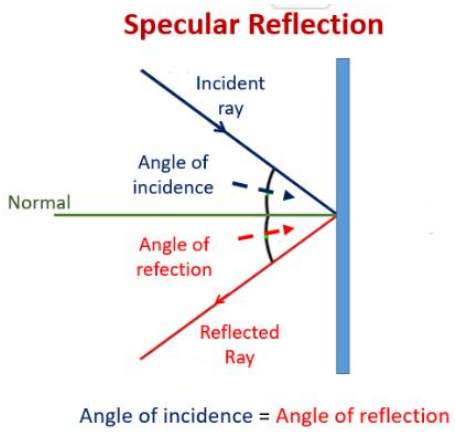



**Rindells Vector models:**

https://odeon.dk/pdf/Asva97.pdf

**Specular Reflection diagram:**

*(OnlineMathLearning[2005])*




**Sabines Formula:**

*(SynAudCon[1996])*

$$T = \frac{0.049V}{S\alpha_S}$$

where $V$ is the volume of the room in cubic feet,

$S$ is the total boundary area (walls, ceiling, and floor),

and $\alpha_S$ is the mean absorption coefficient as defined by Sabine

**Snells Law**

*(PersonalMath[N/A])*

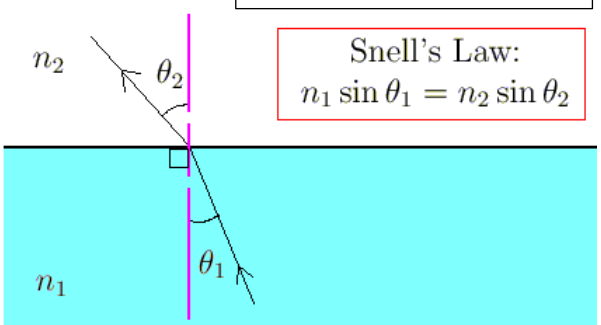



Figure 12 (final tool structure)

```
Acoustic Diagnostic Tool
│
├─ Physics Functions
│   ├─ generate_grid()
│   ├─ compute_pressure_3d_complex()
│   ├─ generate_image_sources_3d()
│   └─ compute_field()
│
├─ Speaker Class
│   ├─ set_position()
│   ├─ set_angle()
│   └─ draw/update_plot()
│
├─ AcousticGUI Class
│   ├─ build_gui() - sliders, text boxes, radio buttons
│   ├─ compute_field() - links GUI input to physics module
│   ├─ update() - updates SPL map
│   ├─ update_room() - handles dimension changes
│   ├─ update_material() - sets reflection coefficients
│   └─ Event Handlers - on_press, on_release, on_motion
│
└─ GUI Launch
    └─ gui = AcousticGUI()
```

# Appendix B: full and final program

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider, TextBox, RadioButtons
import time

# --- Physics functions remain outside the GUI class ---

# constants
c = 343.0  # speed of sound in m/s

# Room setup
room_length = 10.0   # meters X
room_width = 10.0    # meters Y
room_height = 3.0    # meters Z
speaker_height = 1.2
grid_res = 0.2   # coarser resolution for performance

# Reflection parameters
reflection_coeff = 0.7  # 0 = totally absorbent, 1 = totally reflective
max_order_default = 1  # number of reflections (2 works best)

# 3D Grid
def generate_grid():
    global x, y, z, X, Y, Z
    x = np.arange(0, room_length, grid_res)
    y = np.arange(0, room_width, grid_res)
    z = np.arange(0, room_height, grid_res)
    X, Y, Z = np.meshgrid(x, y, z, indexing='ij')

generate_grid()

# Speaker positioning class
class Speaker:
    def __init__(self, position, angle, arrow_length=0.4):
        self.position = position
        self.angle = angle
        self.arrow_length = arrow_length
        self.plot = None

    def draw(self, ax):
        self.plot = ax.arrow(self.position[0], self.position[1],
                             self.arrow_length * np.cos(self.angle),
                             self.arrow_length * np.sin(self.angle),
                             color='cyan', width=0.1, label='Speaker')

    def update_plot(self, ax):
        if self.plot:
            self.plot.remove()
        self.draw(ax)

    def set_position(self, x, y):
        x = np.clip(x, 0, room_length)
        y = np.clip(y, 0, room_width)
        self.position[0] = x
        self.position[1] = y

    def set_angle(self, angle):
        self.angle = angle

# Physics Functions
def compute_pressure_3d_complex(source_pos, source_angle, k, attenuation=1.0):
    dx = X - source_pos[0]
    dy = Y - source_pos[1]
```

```
    dz = Z - source_pos[2]

    distance = np.sqrt(dx**2 + dy**2 + dz**2) + 1e-6
    dir_vector = np.stack((dx, dy, dz), axis=-1)
    dir_unit = dir_vector / distance[..., np.newaxis]

    forward = np.array([np.cos(source_angle), np.sin(source_angle), 0.0])
    cos_angle = (dir_unit[..., 0] * forward[0] +
                 dir_unit[..., 1] * forward[1] +
                 dir_unit[..., 2] * forward[2])

    directivity = ((1.0 + cos_angle) / 2.0) ** 0.5
    phase = np.exp(1j * k * distance)
    air_absorption = np.exp(-0.01 * distance)

    return attenuation * directivity * phase * air_absorption / distance

def generate_image_sources_3d(position, angle, order, current_attentuation=1.0):
    if order == 0:
        return [(position, angle, current_attentuation)]

    images = []

    def reflect(new_position, new_angle, current_attentuation):
        return (new_position, new_angle, current_attentuation * reflection_coeff)

    images.extend([
        reflect(np.array([-position[0], position[1], position[2]]), np.pi - angle, current_attentuation),
        reflect(np.array([2*room_length - position[0], position[1], position[2]]), np.pi - angle, current_attentuation),
        reflect(np.array([position[0], -position[1], position[2]]), -angle, current_attentuation),
        reflect(np.array([position[0], 2*room_width - position[1], position[2]]), -angle, current_attentuation),
        reflect(np.array([position[0], position[1], -position[2]]), angle, current_attentuation),
        reflect(np.array([position[0], position[1], 2*room_height - position[2]]), angle, current_attentuation)
    ])

    if order > 1:
        higher = []
        for pos, ang, atten in images:
            if atten > 0.05:
                higher += generate_image_sources_3d(pos, ang, order-1, atten)
        images += higher

    return images

def compute_field(frequency, direction_angle, max_order, speaker_pos):
    wavelength = c / frequency
    k = 2 * np.pi / wavelength

    p_total = np.zeros_like(X, dtype=complex)
    all_sources = generate_image_sources_3d(speaker_pos, direction_angle, max_order)

    for pos, angle, atten in all_sources:
        if atten > 0.01:
            p_total += compute_pressure_3d_complex(pos, angle, k, atten)

    p_mag = np.abs(p_total)
    spl = 20 * np.log10(p_mag + 1e-12)
    spl = np.clip(spl, -60, 0)
    return spl

# GUI class encapsulating all state
class AcousticGUI:
    def __init__(self):
        self.dragging = False
        self.last_update = 0
        self.update_interval = 0.05

        self.frequency = 200.0
        self.direction_angle = np.deg2rad(45)
        self.max_order = max_order_default

        self.speaker = Speaker(np.array([2.0, 2.0, speaker_height]), self.direction_angle)

        self.listener_height = 1.2
        self.z_index = np.argmin(np.abs(z - self.listener_height))
```

```
    self.build_gui()

def build_gui(self):
    if hasattr(self, 'fig'):
        plt.close(self.fig)

    self.fig = plt.figure(figsize=(12, 6))
    self.ax = self.fig.add_axes([0.3, 0.1, 0.65, 0.85])  # main plot

    self.spl = self.compute_field()
    self.img = self.ax.imshow(self.spl[:, :, self.z_index].T,
                              extent=(0, room_length, 0, room_width),
                              origin='lower',
                              cmap='inferno',
                              aspect='equal')
    plt.colorbar(self.img, ax=self.ax, label='Relative SPL (dB)')
    self.speaker.draw(self.ax)

    self.ax.set_title(f'Wave Interference SPL')
    self.ax.set_xlabel('Room length (m)')
    self.ax.set_ylabel('Room width (m)')
    self.ax.legend()

    # Create side panel axes
    side_panel_x = 0.05
    panel_width = 0.2
    slider_height = 0.03
    spacing = 0.05

    # sliders
    ax_freq = self.fig.add_axes([side_panel_x, 0.8, panel_width, slider_height])
    ax_angle = self.fig.add_axes([side_panel_x, 0.74, panel_width, slider_height])
    ax_order = self.fig.add_axes([side_panel_x, 0.68, panel_width, slider_height])
    ax_listener = self.fig.add_axes([side_panel_x, 0.62, panel_width, slider_height])

    self.s_freq = Slider(ax_freq, 'Freq (Hz)', 20, 20000, valinit=self.frequency)
    self.s_angle = Slider(ax_angle, 'Angle (deg)', 0, 360, valinit=np.rad2deg(self.direction_angle))
    self.s_order = Slider(ax_order, 'Reflections', 0, 3, valinit=self.max_order, valstep=1)
    self.s_listener = Slider(ax_listener, 'Listener Height', 0.1, room_height, valinit=self.listener_height)

    # TextBoxes
    ax_len = self.fig.add_axes([side_panel_x, 0.5, panel_width, slider_height])
    ax_wid = self.fig.add_axes([side_panel_x, 0.44, panel_width, slider_height])
    ax_hei = self.fig.add_axes([side_panel_x, 0.38, panel_width, slider_height])

    self.tb_len = TextBox(ax_len, 'Length', initial=str(room_length))
    self.tb_wid = TextBox(ax_wid, 'Width', initial=str(room_width))
    self.tb_hei = TextBox(ax_hei, 'Height', initial=str(room_height))

    # RadioButtons
    ax_reflect = self.fig.add_axes([side_panel_x, 0.2, panel_width, 0.15])
    self.radio = RadioButtons(ax_reflect,
                              ('plywood', 'Brick wall', 'corrugated iron wall', 'Plaster board'))

    # events
    self.s_freq.on_changed(self.update)
    self.s_angle.on_changed(self.update)
    self.s_order.on_changed(self.update)
    self.s_listener.on_changed(self.update)

    self.tb_len.on_submit(self.update_room)
    self.tb_wid.on_submit(self.update_room)
    self.tb_hei.on_submit(self.update_room)

    self.radio.on_clicked(self.update_material)

    self.fig.canvas.mpl_connect('button_press_event', self.on_press)
    self.fig.canvas.mpl_connect('button_release_event', self.on_release)
    self.fig.canvas.mpl_connect('motion_notify_event', self.on_motion)

def compute_field(self):
    return compute_field(self.frequency, self.direction_angle, self.max_order, self.speaker.position)

def update_room(self, text):
```

```
        global room_length, room_width, room_height
        try:
            room_length = np.clip(float(self.tb_len.text), 2, 100)
            room_width = np.clip(float(self.tb_wid.text), 2, 100)
            room_height = np.clip(float(self.tb_hei.text), 2, 30)
            generate_grid()
            self.z_index = np.argmin(np.abs(z - self.listener_height))
            self.speaker.set_position(self.speaker.position[0], self.speaker.position[1])
            self.img.set_extent((0, room_length, 0, room_width))
            self.ax.set_xlim(0, room_length)
            self.ax.set_ylim(0, room_width)
            new_spl = compute_field(self.frequency, self.direction_angle, self.max_order, self.speaker.position)
            self.img.set_data(new_spl[:, :, self.z_index].T)
            self.fig.canvas.draw_idle()
        except:
            pass

    def update_material(self, label):
        global reflection_coeff
        if label == 'plywood':
            reflection_coeff = 0.7
        elif label == 'Brick wall':
            reflection_coeff = 0.95
        elif label == 'corrugated iron wall':
            reflection_coeff = 1.0
        elif label == 'Plaster board':
            reflection_coeff = 0.9
        self.update(None)

    def update(self, val):
        self.frequency = self.s_freq.val
        self.direction_angle = np.deg2rad(self.s_angle.val)
        self.max_order = int(self.s_order.val)
        self.listener_height = self.s_listener.val
        self.z_index = np.argmin(np.abs(z - self.listener_height))
        self.speaker.set_angle(self.direction_angle)
        self.speaker.update_plot(self.ax)
        new_spl = compute_field(self.frequency, self.direction_angle, self.max_order, self.speaker.position)
        self.img.set_data(new_spl[:, :, self.z_index].T)
        self.ax.set_title(f'Wave Interference SPL (f={self.frequency:.1f} Hz, order={self.max_order})')
        self.fig.canvas.draw_idle()

    def on_press(self, event):
        if event.inaxes != self.ax:
            return
        if event.xdata is None or event.ydata is None:
            return
        dist = np.hypot(event.xdata - self.speaker.position[0], event.ydata - self.speaker.position[1])
        if dist < 0.5:
            self.dragging = True

    def on_release(self, event):
        self.dragging = False

    def on_motion(self, event):
        if not self.dragging or event.inaxes != self.ax:
            return
        current_time = time.time()
        if current_time - self.last_update < self.update_interval:
            return
        self.last_update = current_time
        self.speaker.set_position(event.xdata, event.ydata)
        self.speaker.update_plot(self.ax)
        new_spl = compute_field(self.frequency, self.direction_angle, self.max_order, self.speaker.position)
        self.img.set_data(new_spl[:, :, self.z_index].T)
        self.fig.canvas.draw_idle()

# Launch the GUI
gui = AcousticGUI()
plt.show()
```